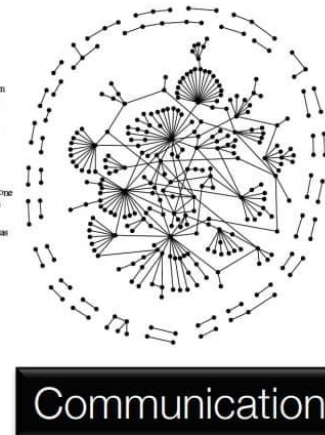
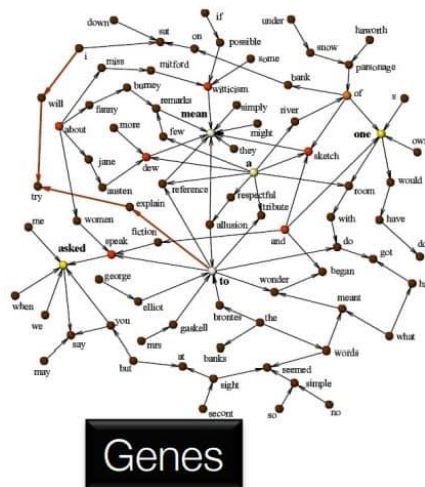
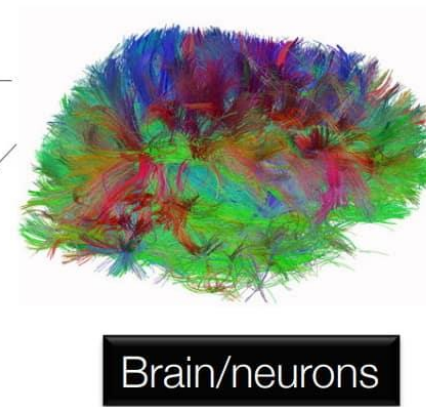
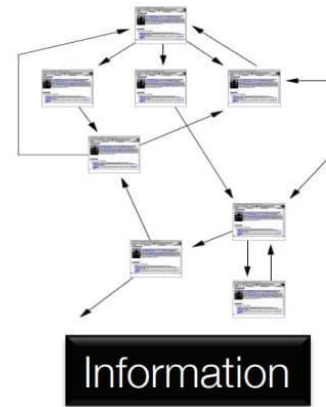
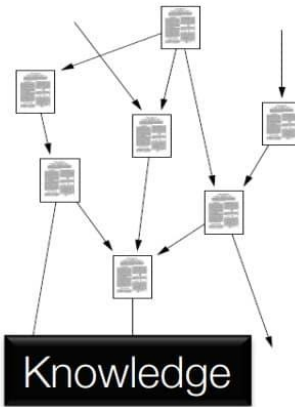
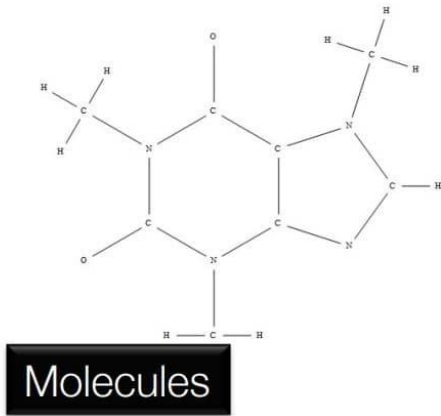
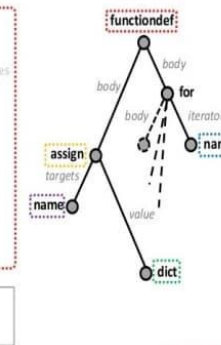


Graph Neural Network

Graph Neural Network



```
def encode(obj):  
    """  
    Encode a (possibly nested)  
    dictionary containing complex values  
    into a form that can be serialized  
    using JSON.  
    """  
    e = {}  
    for key, value in obj.items():  
        if isinstance(value, dict):  
            e[key] = encode(value)  
        elif isinstance(value, complex):  
            e[key] = {'type': 'complex',  
                    'r': value.real,  
                    'i': value.imag}  
    return e  
  
import ast  
tree = ast.parse(" ")
```



Point cloud classification – input data

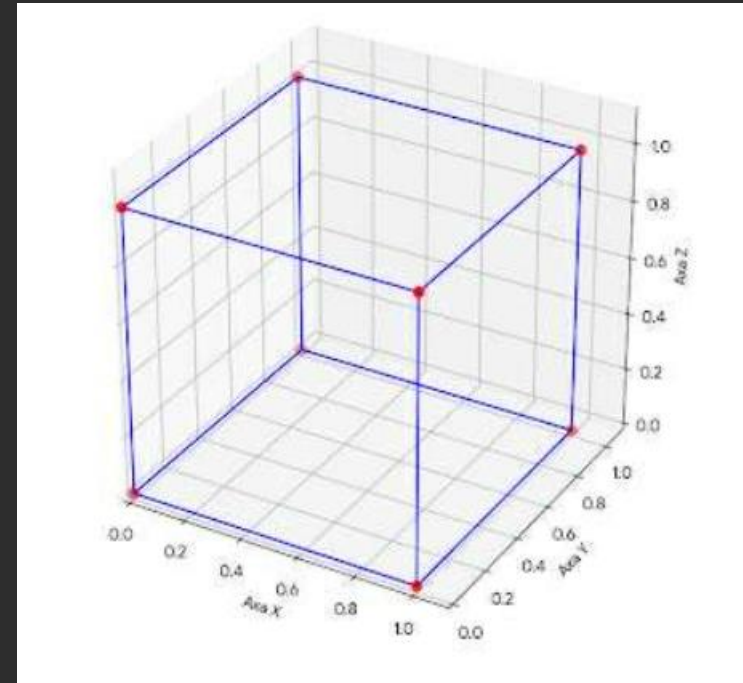
- meshes represented via **pos** and **face** attributes of 40 different 2D and 3d shapes such as cubes, spheres and pyramids.

Point cloud classification – input data: pos

- Pos(positions): the list of vertex positions in space. It's an array of points, each with coordinates(x, y, z)

```
// Coordonatele celor 8 vârfuri ale cubului
// (x, y, z)
pos = [
    // Fața de jos
    [0, 0, 0], // Vârful 0
    [1, 0, 0], // Vârful 1
    [1, 1, 0], // Vârful 2
    [0, 1, 0], // Vârful 3

    // Fața de sus
    [0, 0, 1], // Vârful 4
    [1, 0, 1], // Vârful 5
    [1, 1, 1], // Vârful 6
    [0, 1, 1]  // Vârful 7
];
```



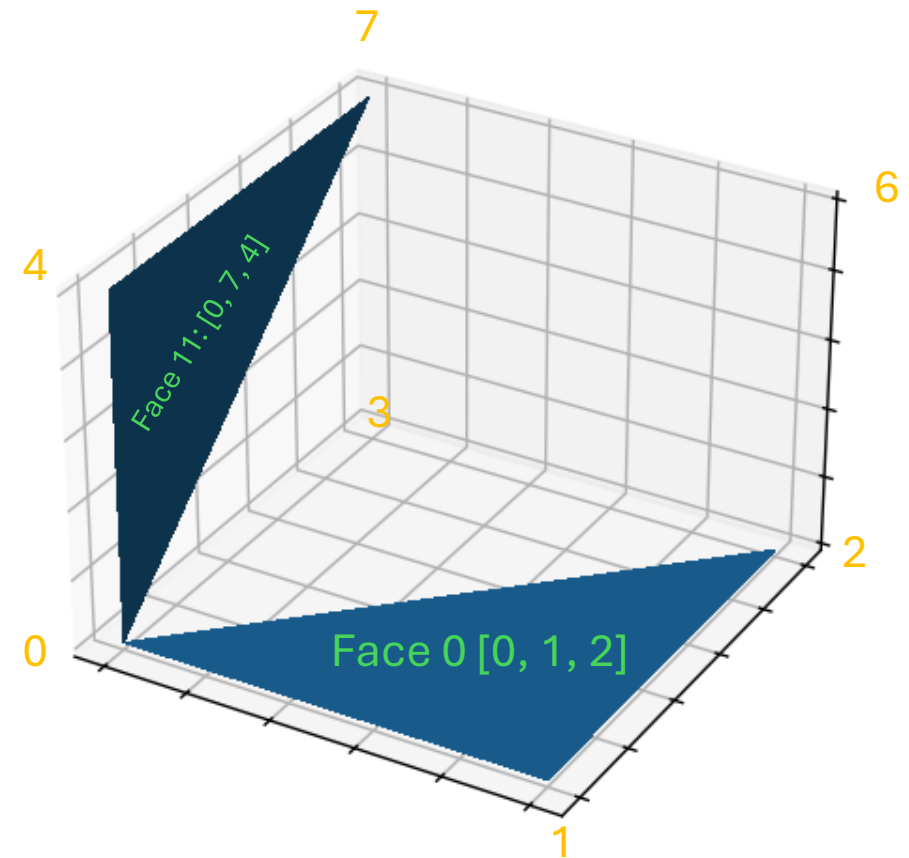
Point cloud classification – input data: faces

- faces represent a flat surface that forms part of the boundary of a solid object.
- a cube has six faces
- faces are typically defined by connecting a set of vertices (points in space) using their indices.
- faces are triangles (3 vertices) or quads (4 vertices), though other polygon types can be used depending on the application or format

Point cloud classification – input data: faces

```
# Coordonatele celor 12 fete ale cubului  
face = [  
    [0, 1, 2], # Face 0: vertices 0, 1, 2  
    [0, 2, 3], # Face 1: vertices 0, 2, 3  
    [4, 5, 6], # Face 2: vertices 4, 5, 6  
    [4, 6, 7], # Face 3: vertices 4, 6, 7  
    [0, 1, 5], # Face 4: vertices 0, 1, 5  
    [0, 5, 4], # Face 5: vertices 0, 5, 4  
    [2, 3, 7], # Face 6: vertices 2, 3, 7  
    [2, 7, 6], # Face 7: vertices 2, 7, 6  
    [1, 2, 6], # Face 8: vertices 1, 2, 6  
    [1, 6, 5], # Face 9: vertices 1, 6, 5  
    [0, 3, 7], # Face 10: vertices 0, 3, 7  
    [0, 7, 4], # Face 11: vertices 0, 7, 4  
]
```

1



Point cloud classification – file data structure

```
data > GeometricShapes > raw > 3d_cube > train > ≡ 3d_cube.off
```

```
1  OFF  # object file format
2  8 6 0 # 8 nr varfuri, 6 nr fețe ( fiecare cu cate 4 varfuri), 0 nr muchii(se pot deduce)
3  1.4730114936828613 -0.3786959350109100 2.5429534912109375 #coordonatele varfurilor
4  1.4730114936828613 -0.3786959350109100 0.5429536104202271 #coordonatele varfurilor
5  1.4730114936828613 -2.3786959648132324 2.5429534912109375 #coordonatele varfurilor
6  1.4730114936828613 -2.3786959648132324 0.5429536104202271 #coordonatele varfurilor
7  -0.5269886255264282 -0.3786959350109100 2.5429534912109375 #coordonatele varfurilor
8  -0.5269886255264282 -0.3786959350109100 0.5429536104202271 #coordonatele varfurilor
9  -0.5269886255264282 -2.3786959648132324 2.5429534912109375 #coordonatele varfurilor
10 -0.5269886255264282 -2.3786959648132324 0.5429536104202271 #coordonatele varfurilor
11 4 0 4 6 2 #coordonate fata - 4 indica ca fața are 4 varfuri
12 4 5 1 3 7 #coordonate fata - 4 indica ca fața are 4 varfuri
13 4 3 2 6 7 #coordonate fata - 4 indica ca fața are 4 varfuri
14 4 1 0 2 3 #coordonate fata - 4 indica ca fața are 4 varfuri
15 4 0 1 5 4 #coordonate fata - 4 indica ca fața are 4 varfuri
16 4 7 6 4 5 #coordonate fata - 4 indica ca fața are 4 varfuri
```

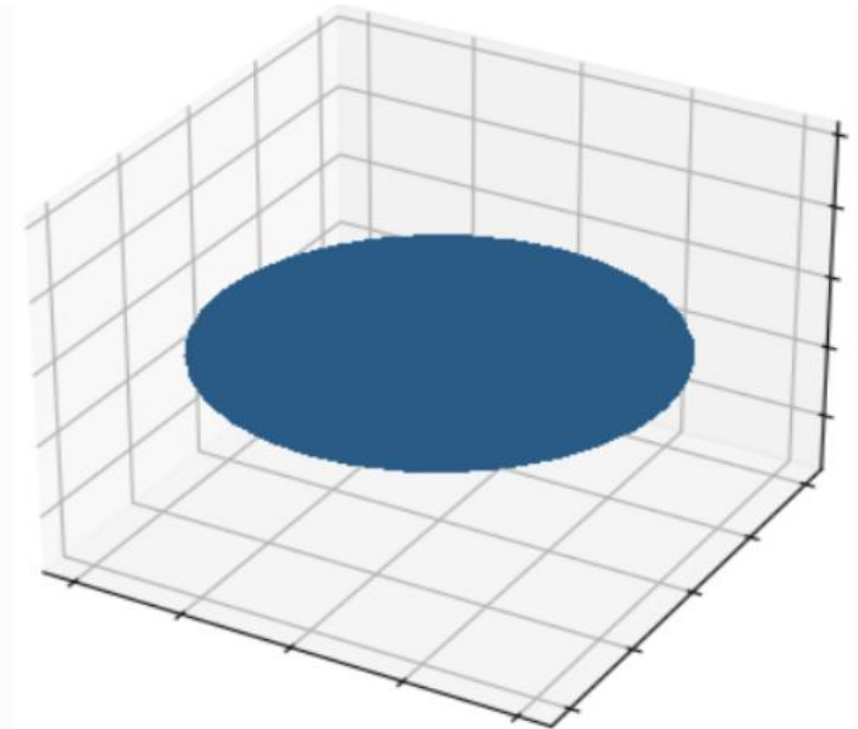
Point cloud classification – reading data

- When visualizing the first mesh in the dataset, we can see that it represents a circle:

```
from torch_geometric.datasets import GeometricShapes

dataset = GeometricShapes(root='data/GeometricShapes')
print(dataset)
>>> GeometricShapes(40)

data = dataset[0]
print(data)
>>> Data(pos=[32, 3], face=[3, 30], y=[1])
```



- Pos = [32, 3] 32 de varfuri, fiecare cu 3 coordonate
- Face = [3, 30] fiecare fata este definita de 3 varfuri(mesh triunghiular), 30 – nr. total de fețe(triunghiuri)
- y = [1] categoria/clasa din care face parte obiectul – pentru GeometricShapes o sa fie intre [0, 39]

Point cloud generation

- Se genereaza 256 puncte pe baza algoritmului Farthest Point Sampling(FPS).

```
import torch_geometric.transforms as T

dataset.transform = T.SamplePoints(num=256)

data = dataset[0]
print(data)
>>> Data(pos=[256, 3], y=[1])
```



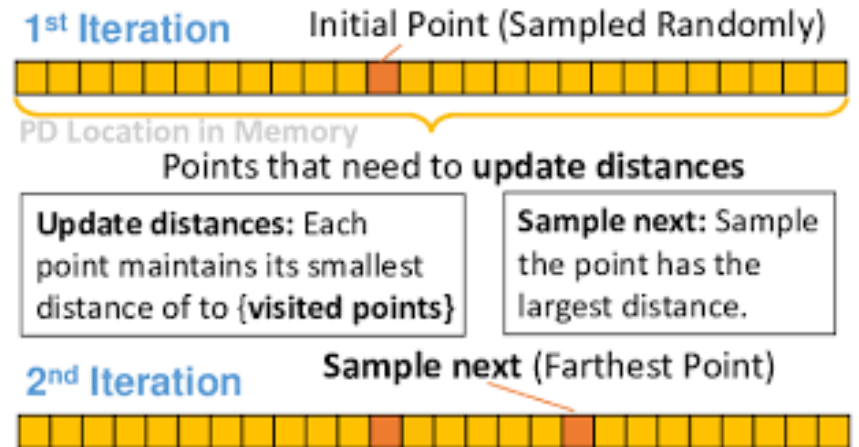
Farthest Point Sampling(FPS)

Algorithm 1 Farthest Point Sampling

Input: $P = \{p_1, p_2, \dots, p_n\}$: point cloud data; k : number of points to sample

Output: $S = \{s_1, s_2, \dots, s_n\}$: sampled point indices

```
1: randomly select initial point:  $s_1 = rand(1, n)$ ;  $S = \{s_1\}$ 
2: for  $i \leftarrow 2$  to  $k$  do
3:   Compute the distance from each point in  $P$  to the set  $S$ :
4:   for  $j \leftarrow 1$  to  $n$  do
5:      $d[j] \leftarrow \text{MinDistance}(p_j, S)$ 
6:   end for
7:   Find the point with the maximum distance:
8:    $s_i \leftarrow \arg \max(d[j])$ 
9:   Add the selected point to the set  $S \leftarrow S \cup \{s_i\}$ 
10: end for
```



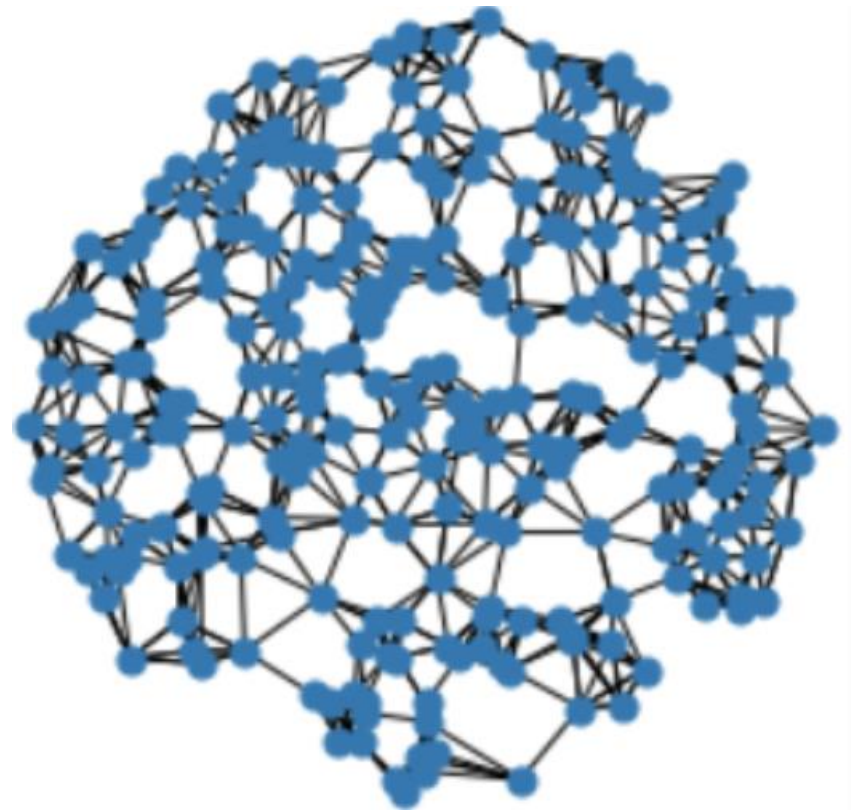
Point Cloud -> graph

- 6 grafuri, fiecare cu 256 de puncte
- folosind k-nearest neighbor search

```
from torch_geometric.transforms import SamplePoints, KNNGraph

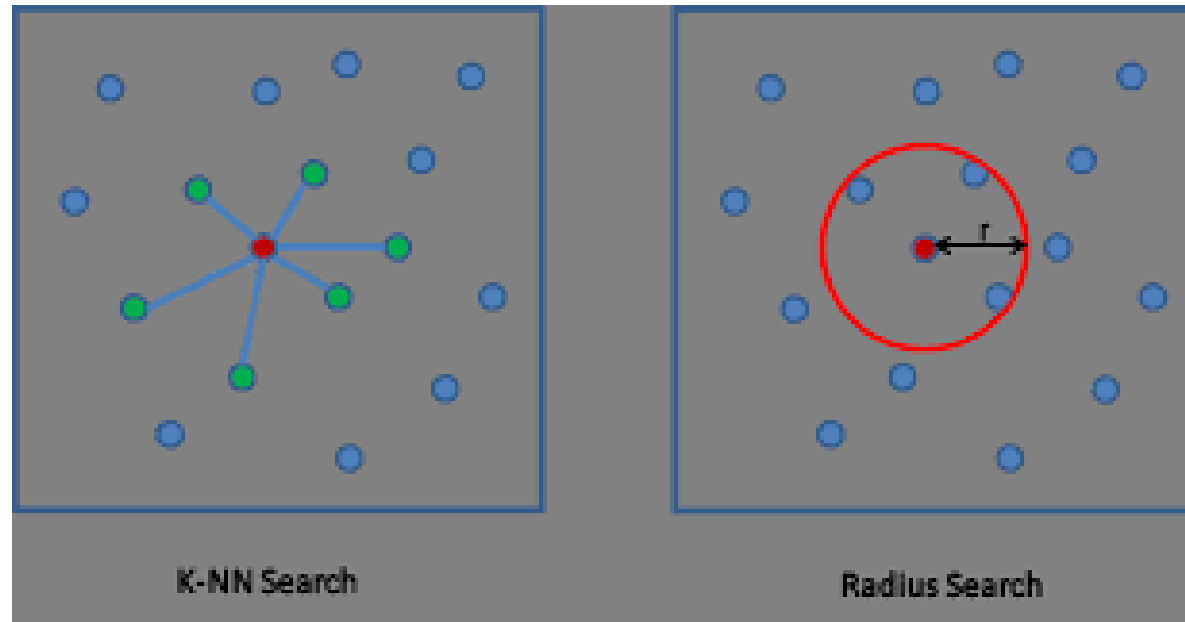
dataset.transform = T.Compose([SamplePoints(num=256), KNNGraph(k=6)])

data = dataset[0]
print(data)
>>> Data(pos=[256, 3], edge_index=[2, 1536], y=[1])
```



k-nearest neighbor search

- Se calculeaza distanta Euclediana sau Manhattan catre cel mai apropiat punct
- <https://www.luseratech.com/knn-visual>



Neighborhood Aggregation

- Se executa un GNN layer care pentru fiecare punct agrega informatia de la vecinii lui directi

$$\mathbf{h}_i^{(\ell+1)} = \max_{j \in \mathcal{N}(i)} \text{MLP} \left(\mathbf{h}_j^{(\ell)}, \mathbf{p}_j - \mathbf{p}_i \right)$$

where

- $\mathbf{h}_i^{(\ell)} \in \mathbb{R}^d$ denotes the hidden features of point i in layer ℓ , and
- $\mathbf{p}_i \in \mathbb{R}^3$ denotes the position of point i .

- Informatia: coordonatele relative intre puncte – offseturi: $p_j - p_i$ pentru fiecare vecin j al punctului i .
- Se aplica MLP pe fiecare vecin al nodului, apoi se ia maximul pentru a identifica acel vecin al carui offset generează pattern-ul pe care MLP-ul a învățat să-l considere relevant (muchie, colț, schimbare de curbură).
- În timpul antrenării, MLP-ul ajustează ponderile astfel încât anumite offseturi (direcții/magnitudini) să scoată în evidență structuri locale

MessagePassing interface

- Interfata MessagePassing din PYG ne ajuta sa definim mesajul pe care vrem sa il propagam prin retea prin implementarea functiei *message()* si sa definim schema de agregare (ex. *max*)

```
class PointNetLayer(MessagePassing):
    def __init__(self, in_channels: int, out_channels: int):
        # Message passing with "max" aggregation.
        super().__init__(aggr='max')

        # Initialization of the MLP:
        # Here, the number of input features correspond to the hidden
        # node dimensionality plus point dimensionality (=3).
        self.mlp = Sequential(
            Linear(in_channels + 3, out_channels),
            ReLU(),
            Linear(out_channels, out_channels),
        )
```

```
def forward(self,
            h: Tensor,
            pos: Tensor,
            edge_index: Tensor,
        ) -> Tensor:
    # Start propagating messages.
    return self.propagate(edge_index, h=h, pos=pos)

def message(self,
            h_j: Tensor,
            pos_j: Tensor,
            pos_i: Tensor,
        ) -> Tensor:
    # h_j: The features of neighbors as shape [num_edges, in_channels]
    # pos_j: The position of neighbors as shape [num_edges, 3]
    # pos_i: The central node position as shape [num_edges, 3]

    edge_feat = torch.cat([h_j, pos_j - pos_i], dim=-1)
    return self.mlp(edge_feat)
```

Update

- la pasul de update se actualiza embedding-ul nodului după agregare.
- se decide ce informatii sa fie trimise ca argumente catre message
- propagate va chema functia message pentru fiecare muchie, va agrega mesajele pe nodul selectat si va apela update pe rezultatul obtinut

```
def forward(self,  
    h: Tensor,  
    pos: Tensor,  
    edge_index: Tensor,  
    ) -> Tensor:  
    # Start propagating messages.  
    return self.propagate(edge_index, h=h, pos=pos)
```

Point cloud classification – output data

- **point cloud classification** - a linear classifier is applied to map the global 32 features per point to one of the 40 classes.
- **segmentation**

```
class PointNet(torch.nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

in_channels: coordonate varfuri

```
        self.conv1 = PointNetLayer(3, 32)
```

```
        self.conv2 = PointNetLayer(32, 32)
```

```
        self.classifier = Linear(32, dataset.num_classes)
```

out_channels: rezultatul agregat trece prin MLP, dând $h_i \in R^{32}$.

Bibliography

[Point Cloud Processing — pytorch_geometric documentation](#)